```
LoggerCallback.py on_epoch_end: Training reached the end of epoch at index 0
LoggerCallback.py on_epoch_end:     with metric loss: 1.6678
LoggerCallback.py on_epoch_end:     with metric masked_categorical_accuracy: 0.41268
LoggerCallback.py on_epoch_end:     with metric val_loss: 1.4746
LoggerCallback.py on_epoch_end:     with metric val_masked_categorical_accuracy: 0.44989
run_utils.py test_transformer: Running text --> text inference on data generator 'Training set (reproducible)'
decoder.py print_predictions_table: ----------------------------------------------------------------------------------
decoder.py print_predictions_table:                       INPUT         TRUE   PRED(MASK)    PRED(GEN)      CORRECT
decoder.py print_predictions_table: ----------------------------------------------------------------------------------
decoder.py print_predictions_table:            453+16+17-N846         1332         1072        <N/A>
decoder.py print_predictions_table:               N95+N72+281          114          108        <N/A>
decoder.py print_predictions_table:               N54-61-N299          184          102        <N/A>
decoder.py print_predictions_table:  987-55+N485-N16-N737-N64         1264         1008        <N/A>
decoder.py print_predictions_table:               N758-N242+90         N426         N207        <N/A>
run_utils.py test_transformer: Running text --> text inference on data generator 'Validation set'
decoder.py print_predictions_table: ----------------------------------------------------------------------------------
decoder.py print_predictions_table:                       INPUT         TRUE   PRED(MASK)    PRED(GEN)      CORRECT
decoder.py print_predictions_table: ----------------------------------------------------------------------------------
decoder.py print_predictions_table:   N942-N85-458-779-N913        N1181        N1228        <N/A>
decoder.py print_predictions_table:      727+N65+611-803+53          523          677        <N/A>
decoder.py print_predictions_table:    N854-397+285+N82+481         N567         N172        <N/A>
decoder.py print_predictions_table:    N363+N71-N790-N91+17          464          472        <N/A>
decoder.py print_predictions_table:  N21+N789+N305-N16+N994        N2093        N1070        <N/A>
```

Figure 4: `Baseline Pre-Norm` model predictions after 1 training epoch.

```
LoggerCallback.py on_epoch_end: Training reached the end of epoch at index 436
LoggerCallback.py on_epoch_end:     with metric loss: 0.22902
LoggerCallback.py on_epoch_end:     with metric masked_categorical_accuracy: 0.92107
LoggerCallback.py on_epoch_end:     with metric val_loss: 0.31508
LoggerCallback.py on_epoch_end:     with metric val_masked_categorical_accuracy: 0.87272
run_utils.py test_transformer: Running text --> text inference on data generator 'Training set (reproducible)'
decoder.py print_predictions_table: ----------------------------------------------------------------------------------
decoder.py print_predictions_table:                       INPUT         TRUE   PRED(MASK)    PRED(GEN)      CORRECT
decoder.py print_predictions_table: ----------------------------------------------------------------------------------
decoder.py print_predictions_table:            453+16+17-N846         1332         1339        <N/A>
decoder.py print_predictions_table:               N95+N72+281          114          114        <N/A>          X
decoder.py print_predictions_table:               N54-61-N299          184          184        <N/A>          X
decoder.py print_predictions_table:  987-55+N485-N16-N737-N64         1264         1263        <N/A>
decoder.py print_predictions_table:               N758-N242+90         N426         N426        <N/A>          X
run_utils.py test_transformer: Running text --> text inference on data generator 'Validation set'
decoder.py print_predictions_table: ----------------------------------------------------------------------------------
decoder.py print_predictions_table:                       INPUT         TRUE   PRED(MASK)    PRED(GEN)      CORRECT
decoder.py print_predictions_table: ----------------------------------------------------------------------------------
decoder.py print_predictions_table:   N942-N85-458-779-N913        N1181        N1184        <N/A>
decoder.py print_predictions_table:      727+N65+611-803+53          523          523        <N/A>          X
decoder.py print_predictions_table:    N854-397+285+N82+481         N567         N568        <N/A>
decoder.py print_predictions_table:    N363+N71-N790-N91+17          464          464        <N/A>          X
decoder.py print_predictions_table:  N21+N789+N305-N16+N994        N2093        N2092        <N/A>
```

Figure 5: `Baseline Pre-Norm` model predictions after training.

```
LoggerCallback.py on_epoch_end: Training reached the end of epoch at index 0
LoggerCallback.py on_epoch_end:     with metric loss: 1.9568
LoggerCallback.py on_epoch_end:     with metric masked_categorical_accuracy: 0.30086
LoggerCallback.py on_epoch_end:     with metric val_loss: 1.864
LoggerCallback.py on_epoch_end:     with metric val_masked_categorical_accuracy: 0.3166
run_utils.py test_transformer: Running text --> text inference on data generator 'Training set (reproducible)'
decoder.py print_predictions_table: ----------------------------------------------------------------------------------
decoder.py print_predictions_table:                       INPUT         TRUE   PRED(MASK)    PRED(GEN)      CORRECT
decoder.py print_predictions_table: ----------------------------------------------------------------------------------
decoder.py print_predictions_table:   99544+13918+N666-17283+N48069        47444        10000        <N/A>
decoder.py print_predictions_table:            4856-5181+N94541-N545        N94321        N1000]        <N/A>
decoder.py print_predictions_table:               N1560+987+553-610         N630         N1000        <N/A>
decoder.py print_predictions_table:  4368+N6816+N9914-7805+N3573...        N55048        N10009        <N/A>
decoder.py print_predictions_table:             4022+6351+7611+8515        26499        10000        <N/A>
run_utils.py test_transformer: Running text --> text inference on data generator 'Validation set'
decoder.py print_predictions_table: ----------------------------------------------------------------------------------
decoder.py print_predictions_table:                       INPUT         TRUE   PRED(MASK)    PRED(GEN)      CORRECT
decoder.py print_predictions_table: ----------------------------------------------------------------------------------
decoder.py print_predictions_table:  35752+N4588+8999-9138-N2792...        59732        10006        <N/A>
decoder.py print_predictions_table:    4435+291-N6497+6482-801-N957        17861        10000        <N/A>
decoder.py print_predictions_table:  226+N46196-N15812+N178-N860...       N105840       N10000]        <N/A>
decoder.py print_predictions_table:  99469+206+77863+N3927-7950+180       165841       100000        <N/A>
decoder.py print_predictions_table:  6152+N727+N461+4374-N36719-538        45519        10000        <N/A>
```

Figure 6: `Large Pre-Norm` model predictions after 1 training epoch.

```
LoggerCallback.py on_epoch_end: Training reached the end of epoch at index 96
LoggerCallback.py on_epoch_end:     with metric loss: 1.1027
LoggerCallback.py on_epoch_end:     with metric masked_categorical_accuracy: 0.56561
LoggerCallback.py on_epoch_end:     with metric val_loss: 1.0925
LoggerCallback.py on_epoch_end:     with metric val_masked_categorical_accuracy: 0.56805
run_utils.py test_transformer: Running text --> text inference on data generator 'Training set (reproducible)'
decoder.py print_predictions_table: ----------------------------------------------------------------------------------
decoder.py print_predictions_table:                       INPUT         TRUE   PRED(MASK)    PRED(GEN)      CORRECT
decoder.py print_predictions_table: ----------------------------------------------------------------------------------
decoder.py print_predictions_table:   99544+13918+N666-17283+N48069        47444        47888        <N/A>
decoder.py print_predictions_table:            4856-5181+N94541-N545        N94321        N94080        <N/A>
decoder.py print_predictions_table:               N1560+987+553-610         N630         N403        <N/A>
decoder.py print_predictions_table:  4368+N6816+N9914-7805+N3573...        N55048        N54808        <N/A>
decoder.py print_predictions_table:             4022+6351+7611+8515        26499        26008        <N/A>
run_utils.py test_transformer: Running text --> text inference on data generator 'Validation set'
decoder.py print_predictions_table: ----------------------------------------------------------------------------------
decoder.py print_predictions_table:                       INPUT         TRUE   PRED(MASK)    PRED(GEN)      CORRECT
decoder.py print_predictions_table: ----------------------------------------------------------------------------------
decoder.py print_predictions_table:  35752+N4588+8999-9138-N2792...        59732        59808        <N/A>
decoder.py print_predictions_table:    4435+291-N6497+6482-801-N957        17861        17880        <N/A>
decoder.py print_predictions_table:  226+N46196-N15812+N178-N860...       N105840       N105808        <N/A>
decoder.py print_predictions_table:  99469+206+77863+N3927-7950+180       165841       165888        <N/A>
decoder.py print_predictions_table:  6152+N727+N461+4374-N36719-538        45519        45888        <N/A>
```

Figure 7: `Large Pre-Norm` model predictions after training.

A.4 Train/test specifications

Table 3 shows the N and L parameters used for the `Baseline` and `Alternate` experiments. We also show the number of datapoints, and the per-datapoint sampling probability. This is a range, with higher probabilites for the simpler sums. Table 4 shows the task specification for the `Large` model variation, which is trained on a more complex setting. We also perform a scan over model size and learning rate to compare the training stability of `Pre-Norm` and `QKV-Norm`. These experiments were performed using an earlier problem configuration shown in Table 5.

Dataset	N	L	Num datapoints	Datapoint probability
Train	$[3, 4, 6]$	$[2, 3]$	110M, acc=90% @ 40M	2×10^{-9} to 5×10^{-24}
Validation	$[5]$	$[2, 3]$	6.4k	1×10^{-14} to 1×10^{-19}
In-distribution	$[3, 4, 6]$	$[2, 3]$	128k	2×10^{-9} to 5×10^{-24}
OOD (interpolation)	$[5]$	$[2, 3]$	128k	1×10^{-14} to 1×10^{-19}
OOD (extrapolation)	$[7, 8, 9]$	$[2, 3]$	128k	7×10^{-21} to 1×10^{-35}

Table 3: Dataset configurations used for `Baseline` and `Alternate` results.

Dataset	N	L	Num datapoints	Datapoint probability
Train	$[4, 5, 7, 8]$	$[3, 4, 5]$	25M	4×10^{-17} to 3×10^{-49}
Validation	$[6]$	$[3, 4, 5]$	6.4k	2×10^{-24} to 2×10^{-36}
In-distribution	$[4, 5, 7, 8]$	$[3, 4, 5]$	128k	4×10^{-17} to 3×10^{-49}
OOD (interpolation)	$[6]$	$[3, 4, 5]$	128k	2×10^{-24} to 2×10^{-36}
OOD (extrapolation)	$[9, 10, 11]$	$[3, 4, 5]$	128k	3×10^{-37} to 3×10^{-67}

Table 4: Dataset configurations used for `Large` results.

Dataset	N	L	Datapoint probability
Train set	$[3, 4, 5, 7]$	$[3, 4, 5]$	4×10^{-13} to 3×10^{-43}
In-distribution	$[3, 4, 5, 7]$	$[3, 4, 5]$	4×10^{-13} to 3×10^{-43}

Table 5: Dataset configurations used for training stability results (Figure 24).

We halt training according to wall time, which leads to a range of observed dataset sizes. Model convergence may also occur much earlier. We therefore show an order-of-magnitude estimate for the number of observed datapoints, as well as the point at which the baseline model reaches 90% per-token accuracy (this represents almost-convergence, which is logged at 92.1%).

Note that our data-generation strategy does not ensure that training examples are exclusive (there may be repetitions), nor that the in-distribution test set does not contain overlap with training examples. The final column is therefore important, because it demonstrates that the highest per-datapoint sampling probability is 2×10^{-9}, whilst the model converges with $\mathcal{O}(10^7)$ datapoints and observes $\mathcal{O}(10^8)$ in total. Since the datapoint probability is 2×10^{-9} for the simplest configurations and 5×10^{-25} for the most complicated, this ensures that the in-distribution evaluation metric is dominated by novel examples. The validation set is only used for visual inspection of model behaviour during training, as in Figures 4-7.

A.5 Model specification (main experiments)

We use a decoder architecture, meaning that the dot-product self-attention layers are causally masked such that token t can only attend to tokens $\leq t$. The model has the following structure:

$$\begin{array}{c}
\texttt{Embedding + positional encoding} \\
\downarrow \\
N_{layer} \times \left[\begin{array}{c} \texttt{Attention block} \\ \downarrow \\ \texttt{Feed-forward block (ReLU)} \end{array}\right] \\
\downarrow \\
\texttt{Multi-layer perceptron (ReLU)} \\
\downarrow \\
\texttt{Predicted logits}
\end{array}$$

Embedding + positional encoding We initialise each token embedding as $x = x_{type} + x_{pos}$, where x_{type} is a token embedding with N_{emb} elements, and x_{pos} use cyclic positional encodings of the same form as the original transformer architecture [1], with N_{freq} frequencies initialised as a base e log-series between periods of 3 and $1k$ tokens. For each sequence, all position indices are simultaneously offset by a random integer between 0 and 50. This augmentation is designed to encourage the use of *relative* positions rather than absolute. The frequencies are then left as trainable parameters. The positional encodings contribute the first $2N_{freq}$ components of x_{pos}, and the remaining are set to 0. This configuration guarantees that the token embeddings and positional encodings can be made orthogonal in the first layer, and x_{pos} have constant L_2-norm, consistent with our theoretical structure.

Attention block N_{layer} is the number of residual blocks of our model, where our baseline is $N_{layer} = 10$. The update is as formulated in section 4, where H is the number of parallel attention heads per layer. Since the embeddings have length N_{emb}, we must have $N_x = N_{emb}$, whilst the latent dimension N_{qkv} is configurable. Either the `Pre-Norm` or `QKV-Norm` strategy is used, as configured.

Feed-forward block The feed-forward blocks update embeddings using the function $x \to x + FF(\texttt{LayerNorm}(x))$, where FF is a dense network with one hidden layer of size N_{ff}. The network uses a `ReLU` [65] activation function on the intermediate layer, followed by a linear projection back onto embedding space. To maintain consistency with other models, we apply `LayerNorm` at the input to FF. Both `LayerNorm` and FF use bias parameters.

Multi-layer perceptron The final embeddings x are mapped onto token logits y using the function $y = MLP(\texttt{LayerNorm}(x))$, where MLP is a multi-layer perceptron with two hidden layers of size N_{MLP} and ReLU activation. The final layer is a linear projection onto the space of logits, which has length 17. For the training stability scan in Figure 24, the MLP has three hidden layers instead.

Hyperparameters Table 6 shows the hyperparameters used to configure the networks of the main experiments. Table 7 show the hyperparameters used for the training stability analysis. This experiment also uses encoder-decoder models, following the same setup as the original transformer architecture [1] and with the layer configurations listed here.

Loss The loss function is `categorical cross entropy`, calculated from the output logits.

A.6 Model initialisation

We use a custom initialisation strategy to give control over the initial state of the model. In particular, we use `Checkpoint` layers to ensure that the initial states are comparable between `Pre-Norm` and `QKV-Norm`. This ensures that any observed differences are driven by the normalisation function, rather than being confounded by the layer placement creating more/less favourable initial conditions.

`Checkpoint` layers are calibrated on the first training batch immediately prior to training. They use this data to measure the standard deviation at that point, and calculate a scale factor that fixes the standard deviation to a pre-defined hyperparameter σ. All subsequent passes through the layer simply